

RAG Pipeline - Agentic RAG: Adaptive RAG

1. Definition

Adaptive RAG is a strategy that dynamically adjusts the retrieval approach based on the **complexity** of the user's query.

Unlike standard RAG (which always retrieves) or CRAG (which retrieves then corrects), Adaptive RAG **classifies** the query *first* to determine if retrieval is even necessary, and if so, how much. It moves the decision-making to the very beginning of the pipeline.

2. Core Components

A. Query Analysis (The Classifier)

This is the distinct feature of Adaptive RAG. A smaller, faster LLM (or a router) analyzes the user's input and routes it to one of three strategies:

1. **No Retrieval:** For simple, general knowledge queries (e.g., "Write a python function to print hello"). The LLM answers directly from its training data.
2. **Single-Shot RAG:** For straightforward factual queries (e.g., "What is the refund policy?"). The system performs one standard retrieval and generation step.
3. **Iterative / Agentic RAG:** For complex, multi-hop queries (e.g., "Compare the features of Product A vs. Product B and conclude which is better for enterprise"). The system enters a multi-step loop of retrieval, reasoning, and re-retrieval.

B. Online Testing (Validation Loop)

This refers to the **Self-Correction** mechanism.

- **Concept:** Similar to "Unit Testing" in software, the agent generates an answer and then runs a "test" on it (using an LLM-as-a-Judge).
- **Assertions:**
 - "Is the answer grounded in the facts?" (Faithfulness)
 - "Does the answer address the user's question?" (Relevance)
- **Retry:** If a test fails, the agent automatically "retries"—it might re-write the query, fetch new documents, or re-generate the answer.

3. Specialized Models: Command R

While Adaptive RAG is an *architecture* that can run on any LLM (like GPT-4), specialized models like **Cohere's Command R** are often used to implement it efficiently.

- **Why Command R?**

- **Tool Use Optimization:** It is fine-tuned to select tools (routes) accurately, which is critical for the "Classifier" step.
- **Long Context (128k):** It can ingest massive amounts of retrieved documents in the "Complex" path without crashing.
- **Citations:** It is trained to explicitly cite where it got its information, making the "Validation" step much easier and more reliable.

4. The Workflow (Putting it together)

1. **Input:** User asks a question.
2. **Router (Classifier):** Analyzes complexity.
 - *If Simple:* → LLM Generation → End.
 - *If Moderate:* → Retrieve → Generate → End.
 - *If Complex:* → **Agent Loop:**
 1. Decompose Query.
 2. Retrieve for Part 1.
 3. Retrieve for Part 2.
 4. Synthesize.
 5. **Test/Validate:** Did we answer it?
 6. *If Fail:* Retry loop.
 7. *If Pass:* Final Answer.

5. Key LangChain Tools

Component	Tool / Implementation
Routing	LangGraph : Essential for building the branching logic (Simple vs. Complex paths) and the loops.
Classifier	ChatPromptTemplate (Structured Output) : A prompt that forces the LLM to output a category (simple , moderate , complex).
Validation	PydanticOutputParser : Used to enforce a binary pass / fail grade from the evaluator LLM.
Model	ChatCohere (Command R) : Excellent choice for the driving LLM due to high performance-to-cost ratio and RAG capabilities.